TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 6

ConfigMaps and Secrets

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 6: ConfigMaps and Secrets 3

6.1 Separate config from code 3

6.2 Creating ConfigMaps 4

6.3 Consuming config in a pod 5

6.4 Secrets 5

6.5 Immutable config and the Downward API 6

6.6 Live lab 7

Class 6 - Top 20 interview questions 8

K8S · CLASS 06 · 60-90 minutes

ConfigMaps and Secrets

By the end you can keep configuration and passwords out of your images, inject them as environment variables or files, and explain how to keep Secrets truly secret.

What you will learn today

- Why configuration must not be baked into images
- ConfigMaps: create from literals, files and YAML
- Four ways to consume config: env, envFrom, volume, command arguments
- Secrets: types, base64, stringData, imagePullSecrets
- Updating config: what reloads automatically and what does not
- Immutable ConfigMaps/Secrets and the Downward API
- Securing Secrets: encryption at rest, RBAC, external secret managers

Part	Topic	Time
1	Story and ConfigMaps	15 min
2	Secrets and security	20 min
3	Updates, immutability, Downward API	10 min
4	Live lab	25 min
5	Recap and interview questions	10 min

6.1 Separate config from code

Ravi's story
The QuickBite orders image had the database password and the payment-gateway URL written inside it. To use a different database in testing, Ravi had to rebuild the image. Worse, anyone who pulled the image could read the production password. His team lead quoted the Twelve-Factor App rule: "Store config in the environment."

Build **one image** and run it everywhere - dev, test, prod - with different settings supplied at runtime. Kubernetes gives you two objects for that:

Object	Holds	Example values
ConfigMap	Non-sensitive settings, as key-value pairs or whole files	APP_MODE=prod, LOG_LEVEL=info, nginx.conf, menu.json
Secret	Sensitive data: passwords, tokens, keys, certificates	DB_PASSWORD, API keys, TLS cert, registry login

Everyday example
A ConfigMap is the **notice board** in the office kitchen - anyone can read today's menu and timings. A Secret is the **locker** with a key - only people who are given access can open it. Both can change without rebuilding the office.

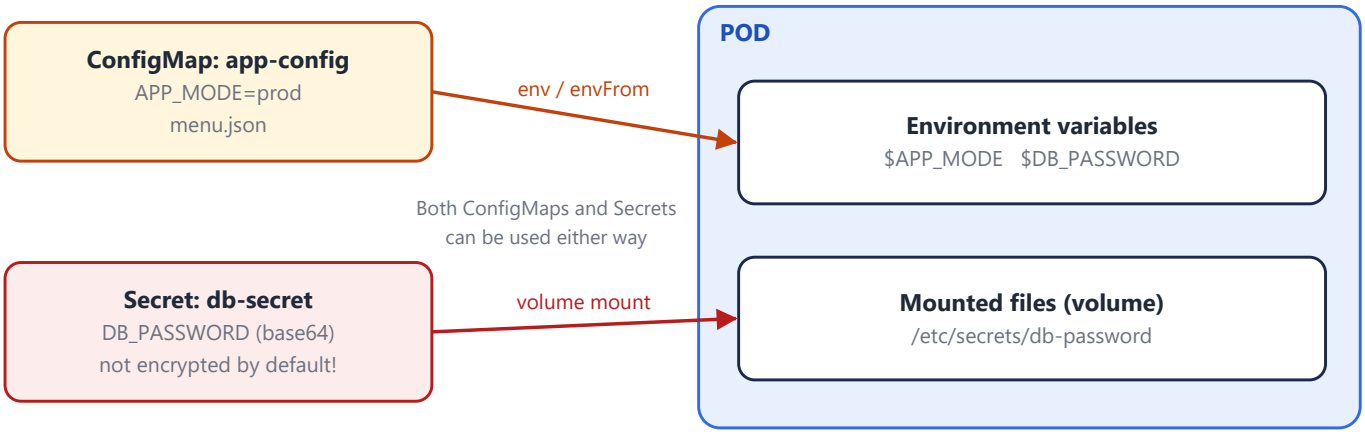


Figure 6.1 - ConfigMaps and Secrets reach a pod as environment variables or files

6.2 Creating ConfigMaps

Imperative

```
kubectl create configmap app-config --from-literal=APP_MODE=prod --from-literal=LOG_LEVEL=info
kubectl create configmap menu-file --from-file=menu.json
kubectl create configmap app-config --from-env-file=app.env --dry-run=client -o yaml
```

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: prod
  LOG_LEVEL: info
  menu.json: |
    {"specials": ["biryani", "masala dosa"]}
```

6.3 Consuming config in a pod

config-demo.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo mode=$APP_MODE pass=$DB_PASSWORD; sleep 3600"]
      env:
        - name: APP_MODE                    # 1. a single key
          valueFrom:
            configMapKeyRef: { name: app-config, key: APP_MODE }
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: { name: db-secret, key: password }
      envFrom:                              # 2. every key as a variable
        - configMapRef: { name: app-config }
      volumeMounts:                         # 3. keys as files
        - name: config-vol
          mountPath: /etc/config
        - name: secret-vol
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: config-vol
      configMap: { name: app-config }
    - name: secret-vol
      secret: { secretName: db-secret }
```

Method	Good for	Updates live?
`env` **with keyRef**	A few specific values	No - restart the pod
`envFrom`	Load all keys at once	No - restart the pod
Volume mount	Config files (nginx.conf, application.properties)	Yes, within about a minute (not with subPath)
Command args `$(VAR)`	Passing values as flags	No - restart the pod

6.4 Secrets

PowerShell

```
kubectl create secret generic db-secret `
  --from-literal=password='S3cr3t!' --from-literal=user=quickbite
kubectl get secret db-secret -o yaml           # values look scrambled... but are they?
kubectl get secret db-secret -o jsonpath='{.data.password}'
# decode in PowerShell:
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("UzNjcjN0IQ=="))
```

Common mistake

base64 is encoding, not encryption. Anyone who can read the Secret object can decode it in one line. Never commit Secret YAML with real values to Git.

Secret type	Used for
Opaque (generic)	Any user-defined data - the default
kubernetes.io/tls	TLS certificate and key, used by Ingress
kubernetes.io/dockerconfigjson	Private registry login, referenced as imagePullSecrets
kubernetes.io/service-account-token	Long-lived ServiceAccount tokens (legacy)
kubernetes.io/basic-auth, ssh-auth	Username/password, SSH keys

In YAML you can use `stringData` to write plain text and let Kubernetes encode it:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  user: quickbite
  password: S3cr3t!
```

How to really protect Secrets

- **Encryption at rest** - configure the API server (EncryptionConfiguration or a cloud KMS) so Secrets are encrypted inside etcd. Managed clusters offer this as a setting.
- **RBAC** - give `get/list` on Secrets only to the ServiceAccounts that need them.
- **Mount as files** rather than env vars where possible; env vars leak into crash dumps and logs.
- **External secret managers** - HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, via the External Secrets Operator or the Secrets Store CSI driver.
- **Safe Git** - Sealed Secrets or SOPS encrypt Secret files so they can live in Git.

6.5 Immutable config and the Downward API

Setting `immutable: true` on a ConfigMap or Secret prevents accidental edits and reduces load on the API server (kubelets stop watching it). To change it, create a new one, e.g. `app-config-v2`, and update the Deployment - which also gives you a clean rollout and rollback.

The **Downward API** lets a pod read facts about itself - its name, namespace, IP, node name, labels and resource limits - as env vars or files, without calling the API server:

```
env:
  - name: POD_NAME
    valueFrom: { fieldRef: { fieldPath: metadata.name } }
  - name: NODE_NAME
    valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
  - name: MEM_LIMIT
    valueFrom: { resourceFieldRef: { resource: limits.memory } }
```

6.6 Live lab

PowerShell

```
kubectl apply -f configmap.yaml
kubectl create secret generic db-secret `
  --from-literal=password='S3cr3t!' --from-literal=user=quickbite
kubectl apply -f config-demo.yaml
kubectl logs config-demo                      # mode=prod pass=S3cr3t!
kubectl exec config-demo -- env | Select-String APP_
kubectl exec config-demo -- ls /etc/config /etc/secrets
kubectl exec config-demo -- cat /etc/config/menu.json

# live update test
kubectl edit configmap app-config             # change LOG_LEVEL to debug, save
kubectl exec config-demo -- cat /etc/config/LOG_LEVEL   # wait ~60 s: becomes debug
kubectl exec config-demo -- printenv LOG_LEVEL          # still info - env never changes

kubectl delete pod config-demo; kubectl delete cm app-config; kubectl delete secret db-secret
```

Try it yourself

- Reference a ConfigMap that does not exist and look at the pod status: **CreateContainerConfigError**. Fix it by creating the ConfigMap.
- For a Deployment, run `kubectl rollout restart deployment/<name>` after changing a ConfigMap used as env vars.

Class 6 recap

- ConfigMap = plain settings; Secret = sensitive data (base64, not encrypted).
- Consume via env, envFrom, volume files or args.
- Volume files update live; env vars need a restart.
- Protect Secrets with encryption at rest, RBAC and external managers.
- Downward API exposes pod facts to the container.

Class 6 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a ConfigMap?

An API object that stores non-confidential configuration as key-value pairs or files, so the same image can run with different configuration in different environments.

Q2. What is a Secret?

An object for small amounts of sensitive data such as passwords, tokens and keys. It is stored base64-encoded and can be protected with RBAC and encryption at rest.

Q3. What is the difference between a ConfigMap and a Secret?

Both inject configuration. Secrets are meant for sensitive data: they are base64-encoded, can be encrypted at rest, are mounted on tmpfs in memory on nodes, and are usually restricted by RBAC.

Q4. Are Kubernetes Secrets secure by default?

Not fully. They are only base64-encoded and stored unencrypted in etcd unless encryption at rest is enabled. Anyone with read access to Secrets or etcd can see them.

Q5. How can you make Secrets more secure?

Enable encryption at rest with a KMS provider, limit access with RBAC, avoid committing them to Git (use Sealed Secrets or SOPS), prefer file mounts over env vars, and use external managers like Vault or cloud secret stores through External Secrets Operator or the CSI driver.

Q6. What are the ways to use a ConfigMap in a pod?

As individual env vars (configMapKeyRef), all keys as env vars (envFrom), as files in a volume, or as command-line arguments via env var substitution.

Q7. If you update a ConfigMap, does the pod see the change?

Volume-mounted files update automatically after a short delay (except subPath mounts). Environment variables do not change until the pod restarts, e.g. via `kubectl rollout restart`.

Q8. What is the difference between data and stringData in a Secret?

`data` expects base64-encoded values; `stringData` accepts plain text and the API server encodes it. stringData is write-only convenience and is merged into data.

Q9. What Secret types exist?

Opaque, kubernetes.io/tls, kubernetes.io/dockerconfigjson, kubernetes.io/basic-auth, kubernetes.io/ssh-auth, kubernetes.io/service-account-token and bootstrap tokens.

Q10. How do you pull an image from a private registry?

Create a docker-registry Secret (`kubectl create secret docker-registry ...`) and reference it in the pod spec under `imagePullSecrets`, or attach it to the ServiceAccount.

Q11. What is an immutable ConfigMap or Secret?

One with `immutable: true`. Its data cannot be changed, which prevents accidental updates and reduces API server watch load. To change it you create a new object and update references.

Q12. What is the size limit of a ConfigMap or Secret?

1 MiB each, because they are stored in etcd. Larger data should go in volumes or external storage.

Q13. What happens if a pod references a missing ConfigMap?

The container cannot start and shows CreateContainerConfigError, unless the reference is marked `optional: true`.

Q14. What is the Downward API?

A way to expose pod and container metadata - name, namespace, IP, node name, labels, annotations, resource requests and limits - to containers as env vars or files.

Q15. How do you create a ConfigMap from a file?

`kubectl create configmap <name> --from-file=<path>`; the file name becomes the key and its content the value. `--from-env-file` turns each line into a key.

Q16. Why should you prefer mounting Secrets as files over env vars?

Env vars can leak through logs, crash reports, `kubectl describe` output or child processes, and they cannot be rotated without a restart. Files on tmpfs can be updated and have tighter permissions.

Q17. How do you decode a Secret value?

`kubectl get secret <name> -o jsonpath='{.data.<key>}'` then base64-decode it. This shows why base64 is not encryption.

Q18. What is the External Secrets Operator?

A controller that reads secrets from external systems such as AWS Secrets Manager, Azure Key Vault, GCP Secret Manager or Vault and creates/updates Kubernetes Secrets automatically.

Q19. What are Sealed Secrets?

A Bitnami tool where you encrypt a Secret with the cluster's public key into a SealedSecret that is safe to store in Git; only the controller in the cluster can decrypt it.

Q20. How do you trigger a rollout when a ConfigMap changes?

Run `kubectl rollout restart`, or put a hash of the ConfigMap in a pod-template annotation (Helm and Kustomize do this) so any change modifies the template and triggers a rolling update.